Technical report: Alpenrecht AI

- **Track:** Track 2B, own project: one legal question → every level of Swiss law
- **Event:** Online
- **Team:** RedPandas: Christopher Yazigi, Giovanni Yazbeck, Alfredo Pio Yazbeck
- **Demo:** web app via `make run` (http://localhost:5055); CLI `make ask`; screenshots in `docs/`

1. Summary

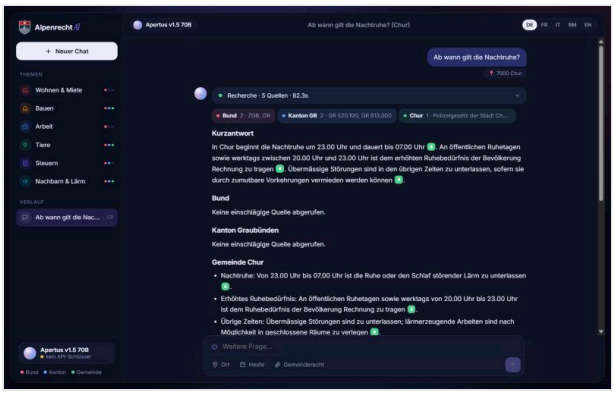
In Switzerland the answer to “Must my dog be on a leash?” or “When does night-time quiet start?” depends on where you live. The rule can sit in federal law, in one of 26 cantonal legal systems or in the regulation of one of more than 2,000 communes, in four languages and on different websites. **Alpenrecht AI** takes a question and an address and returns one answer covering all three levels: it retrieves the federal (Fedlex), cantonal (LexFind) and communal (the commune's own website) rules live, adds leading court decisions (entscheidsuche.ch), and **Apertus-v1.5-70B** writes an answer per level with every claim cited to the official text and **fact-checked three times** before display. It answers in DE / FR / IT / RM / EN, as of any date for federal law, and remembers the last 6 messages of a chat.

On a 100-question benchmark with hand-verified gold sources, **85/100** answers cite the correct source (67 at baseline), cantonal retrieval rises from 11/32 to **29/32**, and answers naming an article that is not a source fall from 13 to **0**. Graded like a lawyer would (0–3), the latest 25-question check averages **2.12/3**, up from 1.72. Fee questions state the amount from the source in **18/18** cases.

2. Architecture

```
question + place + date (+ the last 6 chat messages)
   │
   ├─ swisstopo ──────────── address → commune, BFS no., canton, official languages
   ├─ Apertus (planner) ─── standalone question, levels, SR numbers + articles, title / keyword terms per language
   ├─ Fedlex SPARQL ──────── acts in force → consolidation valid on <date> → articles + annexes (tariffs)
   ├─ LexFind API ────────── Apertus writes the queries, picks the acts, then the articles (26 cantons, one API)
   ├─ communal docs ──────── commune website crawl (2,142 communes) · TLex · readers for 6 large cities · uploads
   ├─ case law ───────────── entscheidsuche.ch: Apertus writes the queries, ≤ 2 leading cases (BGE) with headnotes
   ├─ ranking + filter ───── stemmed BM25 over articles, annexes and tariff tables; Apertus relevance filter
   └─ Apertus (answer) ───── per level, every claim cited [n] ──► 3 verification rounds ──► answer + sources (SSE)
```

- **Apertus steers, code looks up.** Apertus names the likely acts and articles; the code fetches exactly those from the official source. An invented article number simply finds nothing.
- **Three verification rounds** (`verify.py`, `checks.py`), each: deterministic checks (every `[n]` exists, every named article is a source, no prose citations, every CHF amount appears in a source); fetching articles the draft names or the verifier asks for; an Apertus fact-check that lists every claim with a verdict; a rewrite only when the round found something. The UI shows a “3× geprüft” badge with the counts.
- **Communal law without a central collection** (`communal.py`, `cities.py`): BFS number → official website (a table of 2,142 communes built from Wikidata) → regulations page (navigation, sitemap, common CMS paths, embedded JSON lists, TLex) → Apertus picks the regulation and the articles. Readers cover Zürich, Geneva, Basel, Lausanne, Bern and Lugano. Scanned or garbled PDFs are transcribed by Apertus itself (image input, 8B model).
- **Figures:** annexes and tariff tables (“Anhang”, “Annexe”, “Tarif”) are indexed as sources; for fee questions the tariff that belongs to the chosen regulation or federal act is loaded too.
- **Conversation memory:** the planner sees the last `HISTORY_MESSAGES` (default 6) messages, so “and for cats?” is searched as the full question.
- **Cache only while answering:** downloaded law texts and crawls are shared by the parallel steps of a question; `cache/` is emptied as soon as no question is running (uploads are kept).
- **App:** Flask streaming server-sent events to a single-page UI (five languages, light and dark, phones); citation chips show the supporting sentence of the article; a CLI answers in the terminal.

Start page: the three levels for the chosen place	Answer (Chur): cited per level, 3× checked
	

Target architecture: (a) on-premise, or (c) sovereign Swiss cloud. The app is one Docker container (Flask, Python 3.13) that talks to any OpenAI-compatible Apertus endpoint set by `LLM_BASE_URL`: a canton's own vLLM server (on-premise) or the CSCS inference service in Lugano (Swiss cloud, Swiss jurisdiction and data residency). No question or answer leaves that endpoint and the container; nothing is logged. At runtime it reads public Swiss legal sources (Fedlex, LexFind, swisstopo, commune websites, entscheidsuche.ch), so it is not air-gapped; build time needs only PyPI. External runtime dependencies: those public sources, the Apertus endpoint, and the map tiles and fonts loaded by the browser UI.

3. Use of Apertus

- **Models:** `swiss-ai/Apertus-v1.5-70B` for planning, retrieval steering, answers and fact-checks; `Apertus-v1.5-8B` for reading scanned pages (≈ 20 s for a 24-page regulation). Both run on the CSCS OpenAI-compatible endpoint; `LLM_NAME` / `LLM_BASE_URL` / `LLM_API_KEY` follow the judges' convention. The product uses no other model.
- **Other models (evaluation only):** answer grading in §5 (b) and §5 (d) and the drafting of the held-out reference set were done by automatic grading agents (Claude, Anthropic), each checking answers against the official law text. They are not part of the product and never see user questions.
- **How it is used** (prompting only, JSON where structure is needed, a tolerant parser and one retry):
  1. planner: language, a clarified standalone question, levels, SR numbers and articles, search terms;
  2. LexFind agent: act titles and wildcard stems, act choice from up to 25 candidates, article choice from headings (the same heading-based article choice is used for Fedlex);
  3. communal: regulation and article choice; OCR of scanned or garbled PDFs;
  4. case law: search queries and the choice of at most two leading decisions;
  5. relevance filter, then the answer writer (strict citation style, one section per level, next steps);
  6. fact-checker: every claim with a supported / unsupported verdict, missing sources, then a rewrite.
- **Prompts** are in `pipeline.py`, `lexfind_agent.py`, `communal.py`, `caselaw.py` and `verify.py`; answers run at temperature 0.2, planning and checks at 0.

> **Example** (benchmark T071, graded 3/3): "Ab wann gilt die Nachtruhe?" in 7000 Chur → "In Chur beginnt die Nachtruhe um 23.00 Uhr und dauert bis 07.00 Uhr [4]. An öffentlichen Ruhetagen sowie werktags zwischen 20.00 Uhr und 23.00 Uhr ist dem erhöhten Ruhebedürfnis der Bevölkerung Rechnung zu tragen [4]." [4] = Polizeigesetz der Stadt Chur, Art. 30, found on the city's website. 16 claims checked in three rounds.

4. Data

- **Live public sources only:** Fedlex (SPARQL + HTML), LexFind (cantonal law of all 26 cantons), swisstopo (geocoding), the communes' own websites and TLex, entscheidsuche.ch (court decisions).
- `data/communes.json`: commune → official website for **2,142 communes**, built once from Wikidata.
- **Evaluation sets** in `eval/`: 14 questions (first rounds), 100 questions with hand-verified gold sources (68 federal, 32 cantonal; DE 46 / FR 30 / IT 15 / EN 9; 15 cantons), 100 graded test questions, 21 communal questions, random-commune sets and 18 fee questions. All answers, sources and verification logs are stored in `eval/results/`.

- **No personal data** is stored: questions are not logged, the crawl and law-text cache is emptied after each question, and uploaded regulations stay local.

5. Evaluation

(a) Gold-source benchmark (100 questions, live retrieval, `eval/BENCHMARK100.md`):

	Baseline (keyword LexFind, single pass)	Final (Apertus LexFind agent + 3× verification)
Answer cites a gold source	67/100	**85/100**
· federal questions	56/68	57/68
· cantonal questions	11/32	**28/32**
Cantonal gold act retrieved	11/32	**29/32**
Answers naming an article that is not a source	13/100	**0/100**
Prose citations left unconverted · invalid `[n]`	18 · 0	0 · 0
Right language · all headings · disclaimer	99 · 99 · 99	99 · 99 · 99

The verification loop checks about 10 claims per answer and adds 0.4 real sources per answer on average. On the first 14-question set, the 8B model reached the same 13/14 gold citations as the 70B in the final round.

(b) Graded like a lawyer (0 = wrong … 3 = correct, concrete, right sources; real pipeline, 70B, three verification rounds):

Run	Questions	Average /3	Fully correct (3)	Avg time
v1	100	1.55	12	84 s
v2: Apertus picks federal articles, planner examples, no-invented-figures rules, city readers	100	1.72	21	121 s
Latest: fee tariffs, communal relevance filter, verifier fixes (every 4th question)	**25**	**2.12**	**–**	**97 s**

The 100-question runs were graded by separate grading agents; the latest 25-question check was graded by the team, so its gain is best read as a direction.

(c) Communal law and figures.

- **Communal benchmark:** 19/21 questions in 8 communes find and cite the commune's own regulation, e.g. Chur's Polizeigesetz Art. 30 “Nachtruhe 23.00–07.00”. In the graded runs, the communal regulation was retrieved for 9/10 communal questions.
- **Random communes:** the first random set went from 2/10 to 6/10 after the crawler fixes (TLex, two-column PDFs, OCR, polite retries).
- **Fees** (`eval/FEES.md`, 18 questions in 12 communes and federal fees): the short answer states an amount from a source in **18/18** (6/18 before), e.g. Basel dog tax CHF 160, Lugano CHF 115, Zürich parking card Fr. 300, passport CHF 140 (OLDI annex 2), debt-collection extract CHF 17 (GebV SchKG 12a).
- **Robustness:** 0 errors and 0 timeouts in the 100-question runs; right language 99/100.

(d) Independent held-out check (`eval/questions_heldout100.json`, `eval/ACCURACY_heldout100.txt`): 100 new questions written after development, each with a reference answer checked against the official text (49 communal, 12 cantonal, 39 federal; DE/FR/IT/EN). The original prototype and the submitted version were run on the same questions, and graded blind and in pairs, 0–3.

	Average /3	Usable (≥ 2)	Fully correct	Factual error	Off-topic padding
Original prototype	1.38	46	19	53	36
Submitted version	**1.39**	**46**	**21**	**43**	**22**

On unseen questions the submitted version is preferred head-to-head 43–36 (21 ties), makes fewer factual errors (43 vs 53) and pads less, but is not more often fully right. Communal questions remain the hardest (0.90/3 vs 1.95 federal), and the tuning-set figures in (a)–(c) overstate accuracy on new questions. The graders were automatic (single pass), not lawyers.

6. Limitations

- **Answer time:** three verification rounds take an answer to roughly 50–100 s; the UI streams every step meanwhile. `VERIFY_MODEL=swiss-ai/Apertus-v1.5-8B` speeds this up.
- **Federal article choice** is the main lever left: the right act is usually found, the governing article not always.
- **Communal coverage** depends on the commune's website: JavaScript apps, bot blocking or no linked regulations page cannot be crawled; the app then asks for an upload.
- **Scope:** cantonal law is the current version only; case law covers published BGE with an official headnote; article splitting of cantonal PDFs is heuristic. Legal information, not legal advice.

7. Reproducibility

- `make run` (Docker, `python:3.13-slim`) starts the web app; `make test` runs the 11 offline tests (no network, no API); `make ask Q="…" PLACE="…"` answers in the terminal.
- `python eval/bench.py --set questions100.json --tag … --workers 5` re-runs a benchmark live; `eval/build_set100.py` rebuilds and re-verifies the gold set.
- Retrieval is live and Apertus is not perfectly deterministic, so a re-run can differ slightly; every stored run keeps its answers, sources and verification logs in `eval/results/`.

8. Next steps

- **Faster answers:** fact-checking with the 8B model and streaming the final answer.
- **Point-in-time cantonal law** from LexFind's version history.
- **More city platforms** and cantonal court decisions next to the statutes.
- **An on-premise pilot with a canton or commune** for its citizen help desk, with its own regulations loaded.

License

Apache-2.0 (see the repository root). The law texts are official public sources (Fedlex, LexFind, communal websites).

References

[1] Fedlex, the publication platform for federal law: https://www.fedlex.admin.ch · [2] LexFind, cantonal law collections: https://www.lexfind.ch · [3] swisstopo geocoding API · [4] entscheidsuche.ch, Swiss court decisions · [5] Swiss AI Initiative, Apertus: https://huggingface.co/swiss-ai